

Comparative Study of kNN and Weighted kNN Classification on the Palmer Penguins Dataset

Group 6

Ivona Petrović (66m/25) Vojin Đorđević (144m/25)

Contents

1	Introduction	2
2	Problem Description	2
3	Dataset Description	2
4	Exploratory Data Analysis	2
4.1	Summary Statistics	2
4.2	Class Distribution	3
4.3	Feature Correlations	3
4.4	Feature Distributions by Species	3
4.5	Pairwise Feature Relationships	4
4.6	PCA Projection	5
4.7	Outlier Inspection	5
5	Baseline Method: Classic kNN	5
6	Extension Method: Weighted kNN	6
7	Experimental Setup	6
7.1	Train/Test Split	6
7.2	Feature Scaling	6
7.3	Hyperparameter Tuning	6
7.4	Evaluation Metrics	7
8	Results	7
8.1	Hyperparameter Selection	7
8.2	Test-Set Performance	8
8.3	Confusion Matrices	8
8.4	Decision Boundaries	8
8.5	Ablation Study: Effect of Feature Standardisation	9
9	Conclusion	9
	References	10

1 Introduction

This report presents a comparative study of two k-nearest neighbour (kNN) classification methods applied to the Palmer Penguins dataset. The baseline method is classic majority-vote kNN; the extension is inverse-distance weighted kNN, first formalised by Dudani (1976). Both methods are implemented from scratch using NumPy only.

The study is motivated by four research questions: whether weighted kNN outperforms the baseline, what the optimal neighbourhood size is for each method, how much feature standardisation affects performance, and where and why the two methods disagree.

2 Problem Description

The task is multi-class classification: given four continuous morphological measurements of a penguin, predict its species: Adelie, Chinstrap, or Gentoo. The problem is representative of a typical supervised classification setting with moderate class imbalance and partially overlapping feature distributions.

The two methods under comparison differ only in their voting scheme: classic kNN assigns equal weight to each of the k nearest neighbours, while weighted kNN weights each neighbour’s vote by the inverse of its distance to the query point ($w = 1/d$), giving closer neighbours proportionally greater influence.

3 Dataset Description

The Palmer Penguins dataset contains morphological measurements of 344 penguins from three species (Adelie, Chinstrap, Gentoo) observed on three islands in the Palmer Archipelago, Antarctica. After removing 11 rows with missing values, 333 usable samples remain.

Four continuous numeric features are used for classification: bill length (mm), bill depth (mm), flipper length (mm), and body mass (g). Categorical features (island, sex) are excluded to keep the problem focused on morphological measurements. The target variable is species, encoded as integers: Adelie = 0, Chinstrap = 1, Gentoo = 2.

Class counts after cleaning: Adelie 146, Chinstrap 68, Gentoo 119. The imbalance ratio (largest/smallest class) is $146/68 \approx 2.15$, indicating moderate imbalance. This is accounted for by reporting macro-averaged metrics alongside overall accuracy.

4 Exploratory Data Analysis

4.1 Summary Statistics

Table 1 reports mean, standard deviation, minimum, and maximum for each feature grouped by species. Gentoo penguins are consistently larger across all four measurements, particularly in flipper length and body mass. Adelie and Chinstrap overlap more on bill depth, which is the most ambiguous feature for classification.

Table 1: Summary statistics per feature per species (mean $\pm$ std).

Species	Bill length (mm)	Bill depth (mm)	Flipper length (mm)	Body mass (g)
Adelie	38.79 $\pm$ 2.66	18.35 $\pm$ 1.22	189.95 $\pm$ 6.54	3700.66 $\pm$ 458.57
Chinstrap	48.83 $\pm$ 3.34	18.42 $\pm$ 1.14	195.82 $\pm$ 7.13	3733.09 $\pm$ 384.34
Gentoo	47.50 $\pm$ 3.08	14.98 $\pm$ 0.98	217.19 $\pm$ 6.48	5076.02 $\pm$ 504.12

4.2 Class Distribution

Figure 1 shows the number of samples per species after cleaning. Adelie is the most frequent class (146 samples), followed by Gentoo (119) and Chinstrap (68). The moderate imbalance ($\approx 2.15\times$) motivates the use of macro-averaged F1 as a primary evaluation metric alongside accuracy.

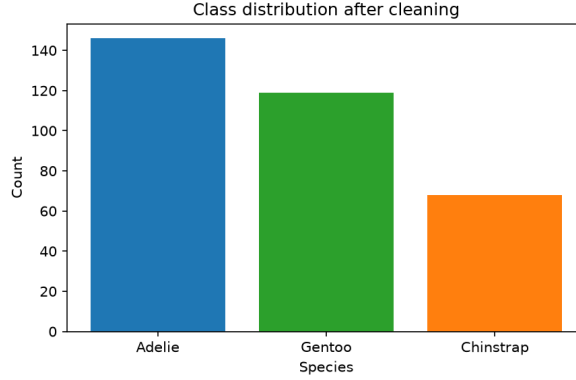


Figure 1: Class distribution after cleaning.

4.3 Feature Correlations

Table 2 shows the pairwise Pearson correlation matrix among the four features. The strongest correlation is between flipper length and body mass ($r = 0.87$), indicating that larger-bodied penguins tend to have proportionally longer flippers. This redundancy is consistent with the high variance captured by the first principal component in the PCA projection.

Table 2: Pairwise Pearson correlation matrix.

	Bill length	Bill depth	Flipper length	Body mass
Bill length	1.00	-0.23	0.65	0.59
Bill depth	-0.23	1.00	-0.58	-0.47
Flipper length	0.65	-0.58	1.00	0.87
Body mass	0.59	-0.47	0.87	1.00

4.4 Feature Distributions by Species

Figure 2 shows overlapping histograms for each of the four features split by species. Bill depth most clearly separates Gentoo from the other two species, while Adelie and

Chinstrap distributions overlap substantially on this feature.

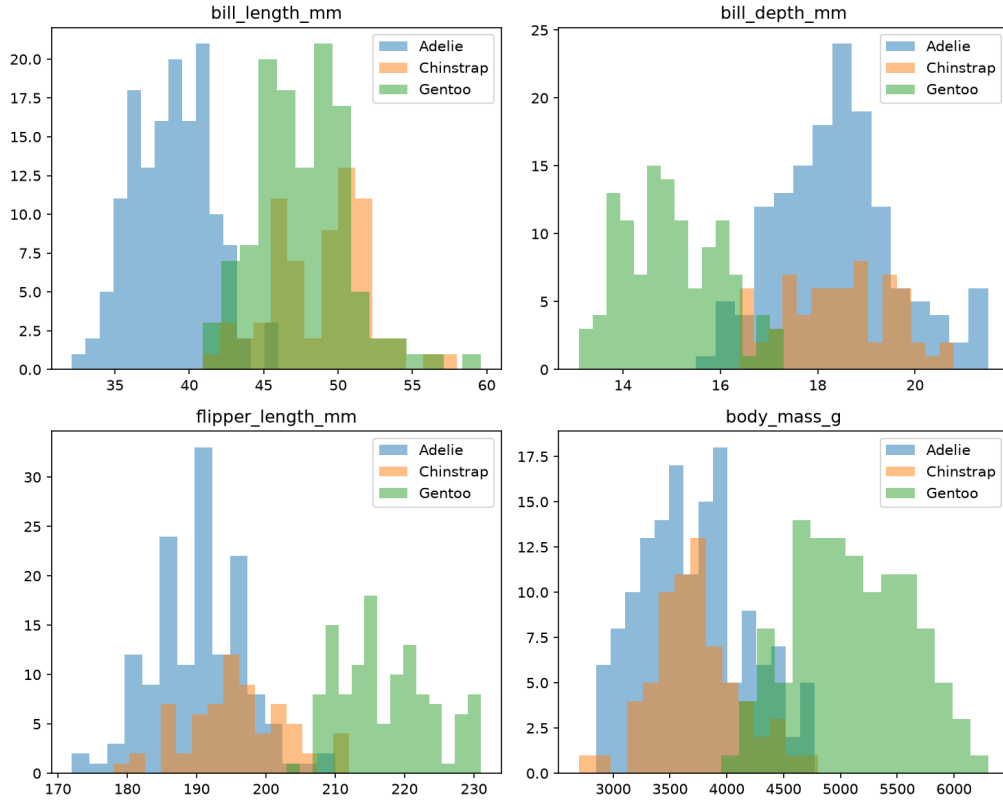


Figure 2: Feature distributions by species.

4.5 Pairwise Feature Relationships

Figure 3 shows scatter plots for the two most informative feature pairs. Flipper length vs. body mass cleanly separates Gentoo from the remaining two species. Bill length vs. bill depth separates Chinstrap from Adelie more clearly than any single feature alone.

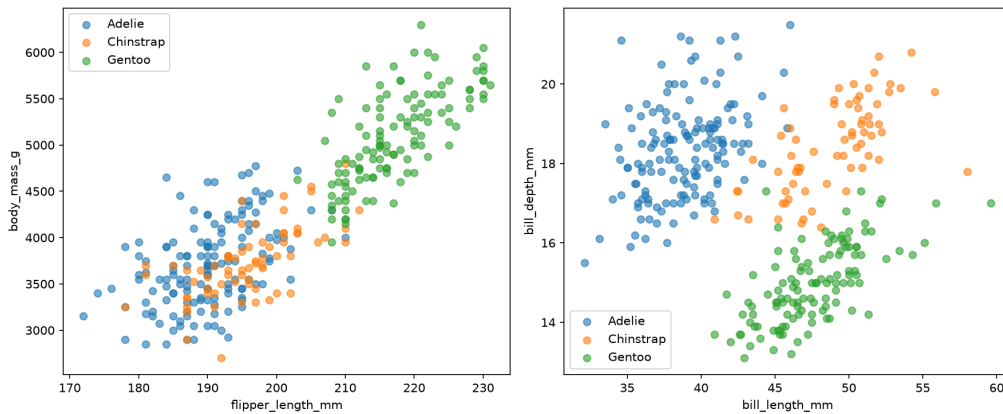


Figure 3: Pairwise scatter plots for the two most informative feature pairs.

4.6 PCA Projection

Figure 4 shows the dataset projected onto its first two principal components, computed on standardised features. The first two components explain **88.2%** of the total variance (PC1: 68.8%, PC2: 19.4%). Gentoo forms a well-separated cluster, while Adelie and Chinstrap show partial overlap in the reduced space, consistent with the histogram analysis above.

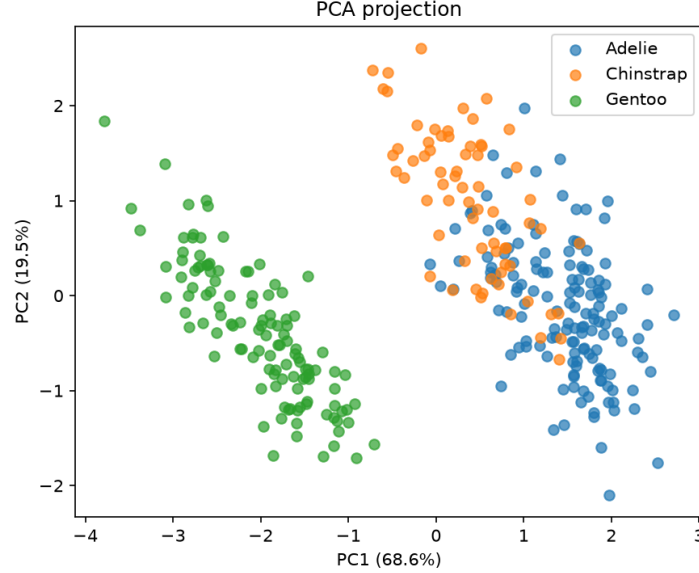


Figure 4: PCA projection onto first two principal components.

4.7 Outlier Inspection

Z-scores were computed per feature on the standardised data. No sample exceeded $|z| > 3$ on any feature, indicating no extreme outliers in the cleaned dataset. No additional rows were removed beyond the missing-value cleaning already performed.

5 Baseline Method: Classic kNN

The k-nearest neighbour classifier assigns a class to a query point by majority vote among its k closest training samples, measured by Euclidean distance. Formally, given a query point $\mathbf{x}$, the predicted class is:

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} \mathbf{1}[y_i = c] \quad (1)$$

where $\mathcal{N}_k(\mathbf{x})$ denotes the set of k nearest neighbours of $\mathbf{x}$ in the training set.

Tie-breaking. If two or more classes receive the same vote count, the class whose neighbours have the smallest summed distance to the query point wins. If still tied, the lowest class index is selected, ensuring deterministic behaviour.

Implementation. The method is implemented from scratch in `knn.py` using NumPy only. Pairwise Euclidean distances are computed via a vectorised matrix multiplication in `distance.py`, avoiding explicit loops over training samples.

6 Extension Method: Weighted kNN

Weighted kNN modifies the voting scheme of classic kNN by assigning each neighbour a weight proportional to the inverse of its distance to the query point. This gives closer neighbours greater influence on the prediction.

The predicted class is:

$$\hat{y} = \arg \max_c \sum_{i \in \mathcal{N}_k(\mathbf{x})} w_i \cdot \mathbf{1}[y_i = c], \quad w_i = \frac{1}{d(\mathbf{x}, \mathbf{x}_i)} \quad (2)$$

where $d(\mathbf{x}, \mathbf{x}_i)$ is the Euclidean distance between the query point and the i -th neighbour.

Zero-distance handling. If the query point exactly matches one or more training points ($d = 0$), the inverse distance is undefined. In this case, only the zero-distance neighbours vote, each with equal full weight, and all other neighbours are ignored, since an exact match should not be diluted by farther points.

Tie-breaking. If two classes accumulate equal total weight, the lowest class index wins. This is rare in practice since weights are continuous values.

Implementation. The method is implemented from scratch in `knn.py`, sharing the same distance computation and batch prediction infrastructure as the baseline.

7 Experimental Setup

7.1 Train/Test Split

The dataset is split into 80% training (266 samples) and 20% test (67 samples) using a stratified split, preserving the class proportions of the full dataset in both subsets. The same random seed (42) is used throughout to ensure reproducibility.

7.2 Feature Scaling

All four features are standardised to zero mean and unit variance using a from-scratch `StandardScaler` implemented in `preprocessing.py`. The scaler is fit on the training data only and applied to both the validation folds and the test set, preventing any leakage of test statistics into the training pipeline.

Feature scaling is essential for kNN because the method relies on Euclidean distance. Without scaling, features with larger numeric ranges (e.g., body mass in grams) would dominate the distance computation, effectively suppressing the contribution of features with smaller ranges (e.g., bill length in millimetres). The effect of standardisation is quantified in the ablation study in Section 8.5.

7.3 Hyperparameter Tuning

The neighbourhood size k is tuned via 5-fold stratified cross-validation over $k = 1, 2, \dots, 20$. Stratified folds are constructed from scratch in `cross_validation.py`: each class's indices are shuffled independently and split evenly across folds, ensuring each fold reflects the overall class distribution.

The generalised `cross_validate(X, y, k_values, method)` harness accepts either `knn_predict` or `weighted_knn_predict` as the `method` argument, so both methods are tuned under identical conditions.

Tie-breaking rule. If two or more values of k achieve the same mean CV accuracy (compared after rounding to six decimal places), the smallest k is selected, favouring the simpler model.

7.4 Evaluation Metrics

Both methods are evaluated on the held-out test set at their respective optimal k . The following metrics are reported:

- Overall accuracy
- Per-class precision, recall, and F1-score
- Macro-averaged precision, recall, and F1-score

All metrics are implemented from scratch in the notebook using the confusion matrix as the primary object. Macro-averaging treats each class equally regardless of size, which is appropriate given the moderate class imbalance noted in Section 3.

8 Results

8.1 Hyperparameter Selection

Figure 5 shows the mean 5-fold CV accuracy for both methods across $k = 1, \dots, 20$. Both methods peak at $k = 4$ with a mean CV accuracy of 0.9879 (std = 0.0060). The curves are nearly identical throughout the entire range, reflecting that on this dataset the two voting schemes rarely produce different predictions.

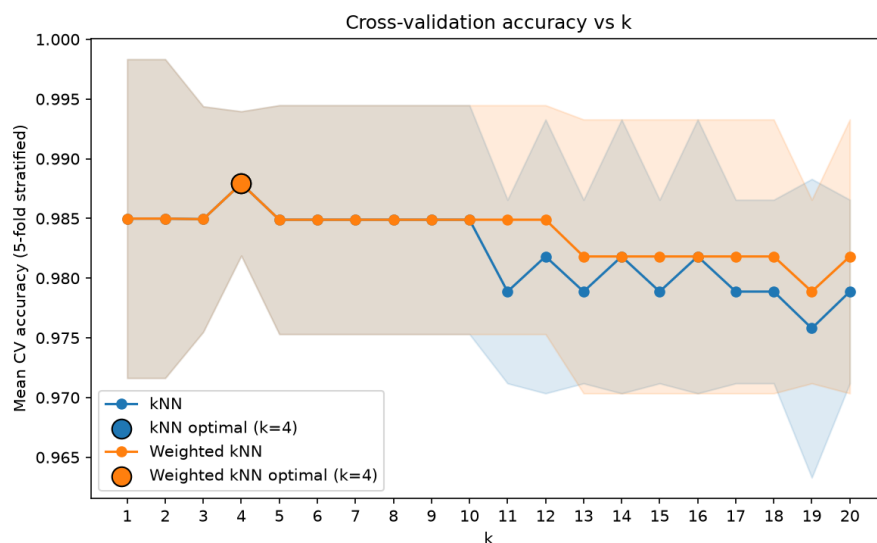


Figure 5: Mean CV accuracy vs. k for both methods. Filled markers indicate the selected optimal k .

8.2 Test-Set Performance

Table 3 reports accuracy, precision, recall, and F1-score for both methods at their optimal $k = 4$ on the held-out test set.

Table 3: Test-set metrics at optimal $k = 4$ for both methods.

Method	Class	Accuracy	Precision	Recall	F1
kNN	Adelie	1.0000	1.0000	1.0000	1.0000
	Chinstrap		1.0000	1.0000	1.0000
	Gentoo		1.0000	1.0000	1.0000
	macro		1.0000	1.0000	1.0000
Weighted kNN	Adelie	1.0000	1.0000	1.0000	1.0000
	Chinstrap		1.0000	1.0000	1.0000
	Gentoo		1.0000	1.0000	1.0000
	macro		1.0000	1.0000	1.0000

Both methods classify all 67 test points correctly, giving perfect scores across every metric. While this may appear suspiciously high, it is consistent with the strong class separability seen in the EDA (Section 4) particularly the clean separation of Gentoo and the clear Adelie/Chinstrap boundary in the pairwise scatter plots and should be read together with the cross-validation results, which average 0.9879 over five folds rather than a single fortunate split.

8.3 Confusion Matrices

Figure 6 shows the confusion matrices for both methods on the test set. Both matrices are perfectly diagonal: all 67 test points are correctly classified, with no misclassifications for either method.

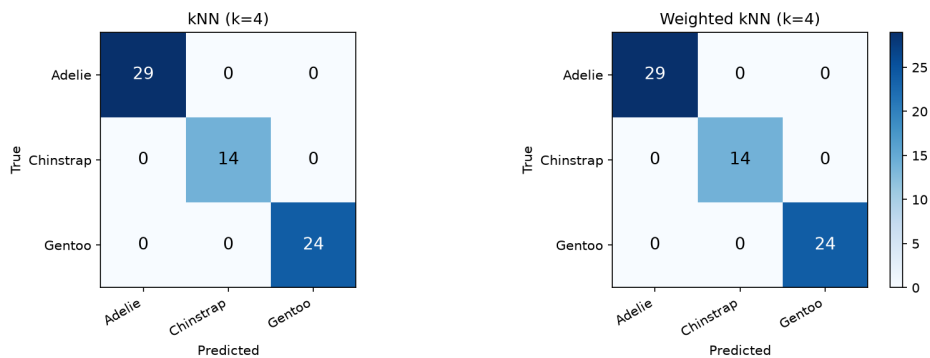


Figure 6: Confusion matrices for kNN (left) and Weighted kNN (right) at optimal $k = 4$.

8.4 Decision Boundaries

Figure 7 shows the decision boundaries of both methods projected onto the first two principal components of the training set. The boundaries are trained on the 2-D PCA projection only, not on the full 4-D feature space, and are included for visual illustration rather than as a performance measure.

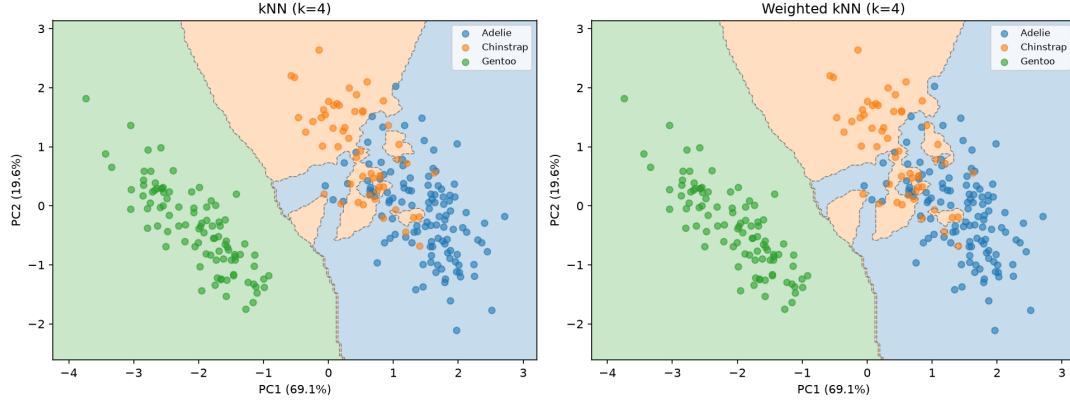


Figure 7: Decision boundaries on the first two PCA components (illustrative only trained on 2-D projection, not full feature space).

8.5 Ablation Study: Effect of Feature Standardisation

Table 4 compares test accuracy for both methods with and without feature standardisation. All other settings are identical: same stratified 80/20 split, same random seed, same 5-fold CV over $k = 1, \dots, 20$. Both methods selected $k = 4$ in every setting, so the comparison isolates scaling as the single variable.

Table 4: Scaled vs. unscaled test accuracy (optimal $k = 4$ in all settings).

Method	Setting	Test Accuracy
kNN	scaled	1.0000
kNN	unscaled	0.7164
Weighted kNN	scaled	1.0000
Weighted kNN	unscaled	0.7015

With unscaled features, both methods still selected $k = 4$ but test accuracy fell sharply: from 1.0000 to 0.7164 for kNN (a drop of 28.4 percentage points) and to 0.7015 for weighted kNN (29.9 points).

9 Conclusion

We compared classic majority-vote kNN with inverse-distance weighted kNN on the Palmer Penguins dataset, addressing the four questions posed in the introduction.

Does weighting improve performance? On this dataset, no measurable difference was observed. Both methods selected $k = 4$, produced near-identical cross-validation curves, and reached identical test performance (1.0000 accuracy). The two voting schemes diverge only when the k nearest neighbours are class-mixed, which rarely occurs given how separable the classes are here.

Optimal neighbourhood size. Both methods peaked at $k = 4$, sitting in a flat, stable region of the CV curve (std = 0.0060) rather than at a sharp, noise-driven peak, indicating the choice is robust rather than a lucky split.

Effect of standardisation. The ablation study shows scaling is the single most impactful preprocessing step. Without it, test accuracy fell from 1.0000 to 0.7164 (kNN)

and 0.7015 (weighted kNN), as body mass came to dominate the Euclidean distance and the Adelie/Chinstrap distinction collapsed.

Where do the methods disagree? They did not disagree on a single test point. This is consistent with the EDA: Gentoo is cleanly separable on flipper length and body mass, and although Adelie and Chinstrap overlap on bill depth, the nearest neighbours at $k = 4$ are unambiguous enough that both schemes reach the same decision.

Limitations. The perfect test accuracy reflects the strong separability of this dataset combined with a small test set (67 samples); it should not be read as evidence that weighting never helps. A noisier or more overlapping dataset, or evaluation over repeated splits rather than a single hold-out, would be needed to distinguish the two methods meaningfully.

References

1. S. A. Dudani, “The distance-weighted k -nearest-neighbor rule,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-6, no. 4, pp. 325–327, 1976.
2. K. B. Gorman, T. D. Williams, and W. R. Fraser, “Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus *Pygoscelis*),” *PLoS ONE*, vol. 9, no. 3, e90081, 2014.